Paradigmas e classificação das linguagens

Nível, geração, paradigma e caminho até a máquina

O tema

Linguagens se classificam por nível (perto da máquina ou do humano), por geração (histórico de abstração) e por paradigma (estilo de pensar o problema). Por baixo de tudo, o processador só entende bits.

Classificação por nível

Linguagem de máquina e baixo nível

Linguagem de máquina é o conjunto de instruções nativas do processador: só 0 e 1. Cada família de CPU (ex.: Intel i3, i5, i7) tem seu instruction set. No exemplo da aula, uma instrução de 16 bits usa 4 bits para o código da operação e 6 bits para cada operando.

Linguagens de baixo nível aproximam-se da máquina e falam com hardware: processador, memória e registradores. Estão ligadas à arquitetura.

> **Resposta da aula**
> As linguagens de baixo nível estão distantes da língua humana escrita. Por isso surgiu o Assembly: mnemônicos (MOV, ADD) no lugar de longas sequências de bits. O assembler (montador) converte Assembly em máquina. Ainda é preciso conhecer registradores e memória.

Alto nível, médio nível e “altíssimo” nível

Alto nível afasta-se da máquina e aproxima-se do humano (em geral, inglês). Não é preciso conhecer registradores. Uma linha vira várias instruções de hardware. Exemplos da aula: ASP, C, C++, C#, Pascal, Delphi, Java, JavaScript, Lua, MATLAB, PHP, Ruby…

Alguns autores chamam C e C++ de médio nível: legíveis, mas com acesso direto a memória. C nasceu para o UNIX, antes escrito em Assembly. Há quem chame Python, Ruby e Elixir de altíssimo nível: muitas bibliotecas e curva mais suave — classificação informal.

Gerações das linguagens

Geração	Nome	Ideia central
1ª	Máquina	Só binário; rápido; difícil
2ª	Assembly	Mnemônicos + assembler
3ª	Procedural / alto nível	Variáveis, if, loop, funções
4ª	Aplicativa	Dizer o quê, não o como (SQL…)
5ª	Declarativa	Lisp, Prolog; IA e ML
6ª*	Alguns autores	Ênfase em redes neurais

3ª geração: procedural — passo a passo. 4ª: não procedural — declara o resultado. SQL é o melhor exemplo da aula. 5ª: declarativa (Lisp, Prolog), ligada a sistemas especialistas, reconhecimento de voz e machine learning.

Arquitetura de Von Neumann

O que é?

Modelo clássico em que programa e dados compartilham a mesma memória, e a CPU busca instruções e dados por um caminho comum. Por isso se fala no “gargalo” de Von Neumann. Aparece nesta aula porque Assembly e baixo nível trabalham perto desse modelo: registradores, endereços e sequência de instruções que a CPU busca e executa.

Paradigmas

Paradigma é o estilo dominante de organizar o programa.

Estruturado

- Fortes nas décadas de 1970 e 1980.
- Evitar desvios incondicionais (goto).
- Refinamentos sucessivos; sequência, condição e repetição.

Orientado a objetos

Objetos com atributos e métodos. Encapsulamento esconde detalhes. Herança especializa. Polimorfismo: a mesma mensagem, comportamentos diferentes. Na aula, Animal tem emitir_som; Gato devolve “miau” e Cachorro “au au”. OO favorece abstração e, segundo a aula, desenvolvimento mais rápido e confiável.

Declarativo: funcional e lógico

- Funcional: o programa é composição de funções; retorno = solução; funções chamam funções; uso intenso de recursão.
- Lógico: base de conhecimento + regras; o motor faz inferência (ex.: Prolog).

Python é multiparadigma: permite estilo procedural/estruturado, OO e trechos funcionais sem forçar um único caminho.

Tradução, compilação e interpretação

Pipeline clássico até o .exe

Código de alto nível → compilador → assembly → montador → código objeto → carregador (onde alocar na memória / objeto relocável) → ligador → executável (.exe). A tradução acontece antes de rodar.

O que é um interpretador?

Programa que lê o código-fonte (ou bytecode) e executa sob demanda, trecho a trecho, sem gerar necessariamente um executável completo no início. Exemplo típico: interpretador Python.

Ideia	Quando vira máquina	Exemplo
Tradução	Termo amplo de conversão	Compilar ou interpretar
Compilação	Antes; gera artefato	C → .exe
Interpretação	Durante a execução	python script.py

AWS CLI é um interpretador?

Não no sentido da aula. AWS CLI é ferramenta de linha de comando: lê comandos, chama APIs da Amazon e devolve respostas. Não é o interpretador de uma linguagem de programação. Pode ser escrita numa linguagem interpretada, mas o papel dela é cliente de nuvem, não tradutor/executor de programa-fonte.

O que não confundir

Baixo nível não é “sempre melhor”: dá controle, mas custa produtividade. Alto nível não é só “preguiça”: uma linha equivale a muitas instruções de máquina. SQL não é procedural como C. AWS CLI não é interpretador de linguagem.

Para levar da aula

Máquina = bits. Assembly = mnemônicos. Alto nível = perto do humano. Gerações 1 a 5 (e menção à 6ª). Von Neumann = programa e dados na mesma memória. Paradigmas: estruturado, OO, funcional e lógico. Python é multiparadigma. Compilar traduz antes; interpretar executa conduzindo o código.

A anotação trouxe níveis, gerações, paradigmas, o pipeline e as perguntas (baixo nível × língua humana, Von Neumann, interpretador, AWS CLI). As respostas foram completadas para o estudo.